COA_Unit-1: Syllabus

Basic Structure of Computers

- Basic organization of computers
- Historical perspective
- Bus structures

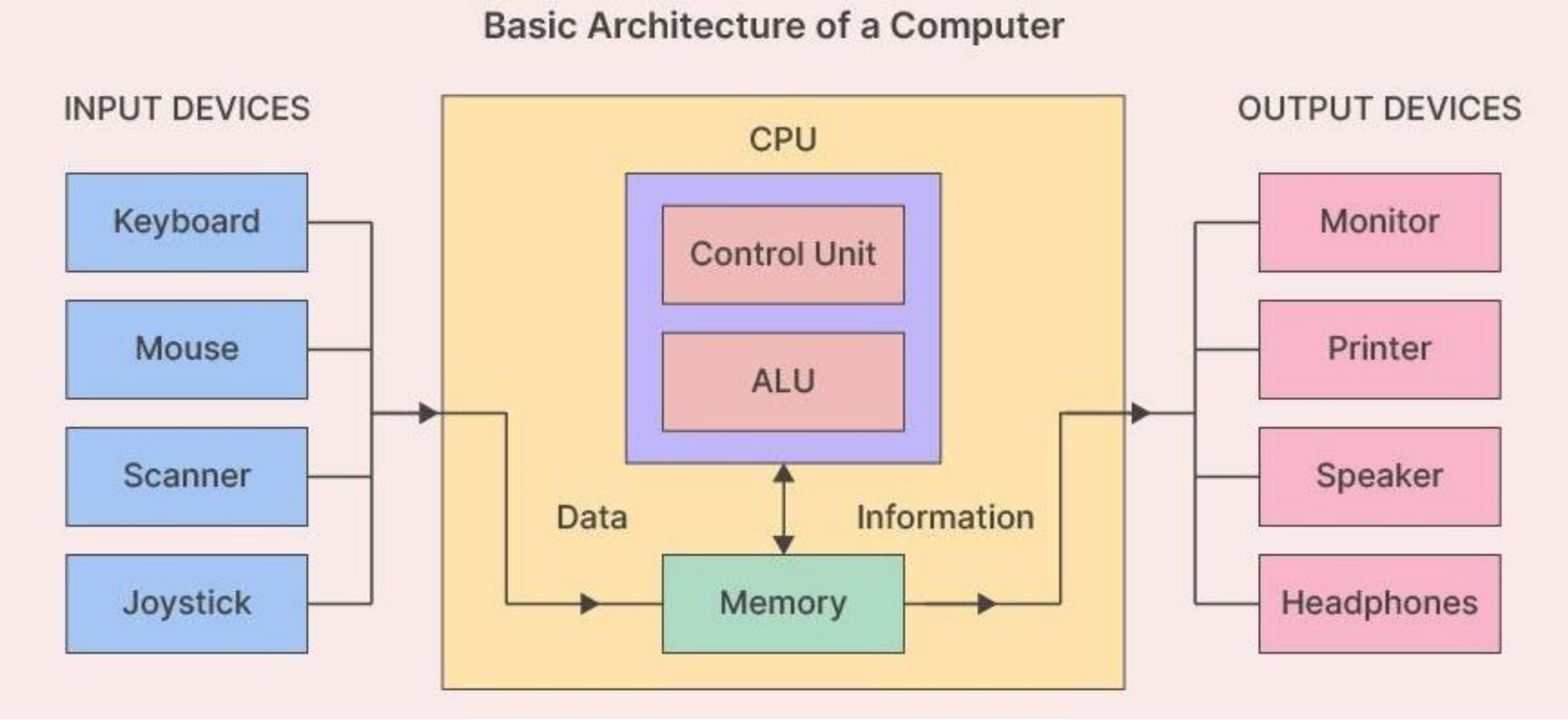
Data Representation

- Data types – Number Systems, ASCII Codes
- Complements
- Fixed point representation
- Floating point representation
- other binary codes
- error detection codes

Computer Arithmetic

- Addition and subtraction
- Multiplication algorithms
- Division algorithms

Basic organization of Computers



Basic organization of Computers cont.,

A computer operates by fetching instructions from memory, processing data (received through input devices or retrieved from memory) using the Central Processing Unit (CPU), and then storing the results in memory or sending them to the outside world through output devices.

Here are some key components and concepts of computer architecture:

1. **Central Processing Unit (CPU)**: The CPU is the brain of the computer responsible for executing instructions. It contains the **arithmetic logic unit (ALU)** for performing arithmetic and logical operations, the **control unit** for instruction decoding and execution, and registers for temporary data storage.

2. **Memory**: Memory is used to store data and instructions that the CPU can access quickly. There are different types of memory in a computer system, including **Random Access Memory (RAM)** for temporary storage and **cache memory** for faster access to frequently used data.

3. **Input/Output (I/O) Devices**: These devices allow users to interact with the computer and exchange data with the outside world. Examples include keyboards, mice, monitors, printers, and network interfaces.

Basic organization of Computers cont.,

4. **Storage Devices:** Storage devices such as hard disk drives (HDDs), solid-state drives (SSDs), and optical drives are used to store data permanently. These devices provide non-volatile storage for applications, documents, and operating system files.

5. **Bus Architecture:** Buses are communication pathways that connect different components of the computer system, allowing them to exchange data and control signals. Common buses include the system bus, which connects the CPU to memory and the expansion bus, which connects peripheral devices to the memory and CPU.

6. **Instruction Set Architecture (ISA):** ISA defines the set of instructions that a CPU can execute and the format of those instructions. Key components involved in this process include the Instruction Register (IR), Program Counter (PC), and general-purpose registers, with the control unit managing the overall execution.

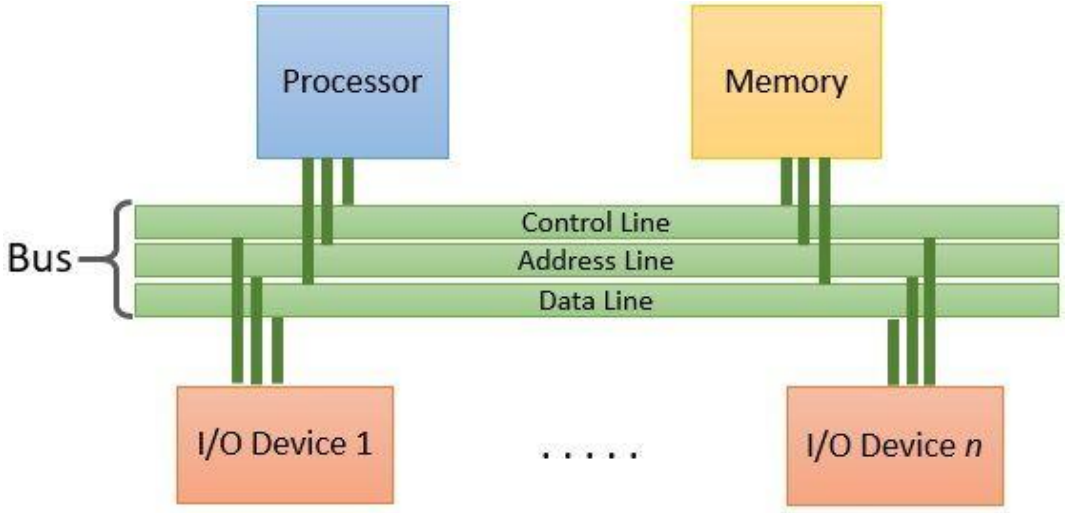
7. **Interrupts:** Interrupts can temporarily halt program execution to address urgent tasks, with the processor saving its state and resuming once the interrupt is serviced.

Historical perspective - Computer Generations

Development of the technologies used to fabricate processors, memories, and I/O units of computers has been divided into five generations:

- **First Generation**, 1945 to 1955: Vacuum tubes (large, slow, and unreliable).
- **Second Generation**, 1955 to 1965: Transistors (smaller, more reliable, and faster).
- **Third Generation**, 1965 to 1975: Integrated Circuits (smaller, faster, more affordable).
- **Fourth Generation**, 1975 to 1990: Microprocessors (personal computers, operating systems, and networking)
- **Fifth Generation** (1990s – Present): (Artificial Intelligence and Parallel Processing)

Bus structure



Bus Structure

Data Lines: Data lines carry and transfer the data among the system components. The data lines are collectively called data bus. A data bus may have 32 lines, 64 lines, 128 lines, or even more lines. The number of lines present in the data bus defines the width of the data bus.

Address Lines: The address lines carry the source or destination of the data present on the data bus. The number of address lines together is referred to as the address bus. The number of address lines in the address bus determines its width.

Control Lines: The address lines and data lines are shared by all the components of the system, so there must be some means to control the use and access of data and address lines. The control signals placed on the control lines control the use and access to the address and data lines of the bus. The control signal consists of the command and timing information. Here the command in the control signal specifies the operation that has to be performed. And the timing information over the control signals specifies when the data and address information is valid.

Data Types

- **Number Systems:** A number system is a way of representing numbers using digits and symbols.
- **Common Number Systems:**
  - Decimal (Base 10): 0–9
  - Binary (Base 2): 0, 1
  - Octal (Base 8): 0–7
  - Hexadecimal (Base 16): 0–9, A–F

Decimal ↔ Binary

- Convert $(345)_{10}$ to binary.
- Convert $(768)_{10}$ to binary.
- Convert $(1025)_{10}$ to binary.
- Convert $(1101011011)_{2}$ to decimal.
- Convert $(10111001101)_{2}$ to decimal.

Decimal ↔ Octal

- Convert $(725)_{10}$ to octal.
- Convert $(1024)_{10}$ to octal.
- Convert $(2560)_{10}$ to octal.
- Convert $(1547)_{8}$ to decimal.
- Convert $(2376)_{8}$ to decimal.

Decimal ↔ Hexadecimal

- Convert $(589)_{10}$ to hexadecimal.
- Convert $(2048)_{10}$ to hexadecimal.
- Convert $(4095)_{10}$ to hexadecimal.
- Convert $(3A7)_{16}$ to decimal.
- Convert $(7F9)_{16}$ to decimal.

Binary ↔ Octal

- Convert $(110101101011)_{2}$ to octal.
- Convert $(101111001101)_{2}$ to octal.
- Convert $(735)_{8}$ to binary.
- Convert $(1247)_{8}$ to binary.

Binary ↔ Hexadecimal

- Convert $(111010110101)_{2}$ to hexadecimal.
- Convert $(101101111001)_{2}$ to hexadecimal.
- Convert $(5A3)_{16}$ to binary.
- Convert $(9F7)_{16}$ to binary.

Fractional Number Conversions

- Convert $(45.625)_{10}$ to binary.
- Convert $(101101.101)_{2}$ to decimal.
- Convert $(72.5)_{8}$ to decimal.
- Convert $(3F.8)_{16}$ to decimal.

ADITYA UNIVERSITY

Binary Coded Numbers

TABLE Binary-Coded Octal Numbers

Octal number	Binary-coded octal	Decimal equivalent	
0	000	0	↑ Code for one octal digit ↓
1	001	1	
2	010	2	
3	011	3	
4	100	4	
5	101	5	
6	110	6	
7	111	7	
10	001 000	8	
11	001 001	9	
12	001 010	10	
24	010 100	20	
62	110 010	50	
143	001 100 011	99	
370	011 111 000	248	

TABLE Binary-Coded Hexadecimal Numbers

Hexadecimal number	Binary-coded hexadecimal	Decimal equivalent	
0	0000	0	↑ Code for one hexadecimal digit ↓
1	0001	1	
2	0010	2	
3	0011	3	
4	0100	4	
5	0101	5	
6	0110	6	
7	0111	7	
8	1000	8	
9	1001	9	
A	1010	10	
B	1011	11	
C	1100	12	
D	1101	13	
E	1110	14	
F	1111	15	
14	0001 0100	20	
32	0011 0010	50	
63	0110 0011	99	
F8	1111 1000	248	

TABLE Binary-Coded Decimal (BCD) Numbers

Decimal number	Binary-coded decimal (BCD) number	
0	0000	↑ Code for one decimal digit ↓
1	0001	
2	0010	
3	0011	
4	0100	
5	0101	
6	0110	
7	0111	
8	1000	
9	1001	
10	0001 0000	
20	0010 0000	
50	0101 0000	
99	1001 1001	
248	0010 0100 1000	

ADITYA UNIVERSITY

Alphanumeric Representation

- Many digital computer applications handle not only numbers but also letters and special characters.
- An **alphanumeric character set** includes decimal digits (0–9), alphabet letters (A–Z), and special symbols such as $, +, and =.
- Such a character set may contain **64 elements** if only uppercase letters are used. It may contain **128 elements** if both uppercase and lowercase letters are included.
- To represent these characters in binary form, **6 bits** are required for up to 64 characters and **7 bits** are required when the character set includes up to 128 characters.
- The standard alphanumeric binary code is ASCII (American Standard Code for Information Interchange). ASCII uses 7-bit binary codes to represent 128 different characters.
- ASCII includes uppercase letters, lowercase letters, decimal digits, and special symbols.
- The binary codes for some characters are listed in standard ASCII table.
- ASCII decimal codes can be converted to binary by removing the three highest decimal bits (011).

TABLE American Standard Code for Information Interchange (ASCII)

Character	Binary code	Character	Binary code
A	100 0001	0	011 0000
B	100 0010	1	011 0001
C	100 0011	2	011 0010
D	100 0100	3	011 0011
E	100 0101	4	011 0100
F	100 0110	5	011 0101
G	100 0111	6	011 0110
H	100 1000	7	011 0111
I	100 1001	8	011 1000
J	100 1010	9	011 1001
K	100 1011		
L	100 1100		
M	100 1101	space	010 0000
N	100 1110	.	010 1110
O	100 1111	(	010 1000
P	101 0000	+	010 1011
Q	101 0001	$	010 0100
R	101 0010	*	010 1010
S	101 0011	)	010 1001
T	101 0100	–	010 1101
U	101 0101	/	010 1111
V	101 0110	,	010 1100
W	101 0111	=	011 1101
X	101 1000		
Y	101 1001		
Z	101 1010		

ADITYA UNIVERSITY

Complements

- For any number system with base *r*, two types of complements exist: **(r−1)'s complement** and **r's complement**.
- In decimal system, these are **9's complement** and **10's complement**.
- In binary system, these are **1's complement** and **2's complement**.

(r−1)'s Complement:

- The (r−1)'s complement of a number is obtained by subtracting each digit from *(r−1)*.
- In decimal, 9's complement is obtained by subtracting each digit from 9.
- In binary, 1's complement is obtained by changing all 0s to 1s and 1s to 0s.

r's Complement:

- The r's complement is obtained by adding 1 to the (r−1)'s complement.
- In decimal, 10's complement = 9's complement + 1.
- In binary, 2's complement = 1's complement + 1.
- 2's complement is the most commonly used representation in computers.

Subtraction Using Complements:

- Subtraction is performed by adding the complement of the subtrahend to the minuend.
- If an **end carry** is generated, it is discarded and the result is positive.
- If no end carry is generated, the result is negative and is represented in complement form.

ADITYA UNIVERSITY

Binary Subtraction using 1's complement

Binary Subtraction using 1's Complement

Positive Result

Problem: $(101101)_2 - (001011)_2$

Solution:

1. Take 1's complement of the subtrahend (001011 → 110100).
2. Add it to the minuend: 101101 + 110100 = 1 100001.
3. End carry is present, so add it to the sum: 100001 + 1 = 100010.

Final Answer: **$(100010)_2$**

Explanation:

Since an end carry is obtained, the result is positive and correct.

Binary Subtraction using 1's Complement

Negative Result

Problem: $(010110)_2 - (101001)_2$

Solution:

1. Take 1's complement of the subtrahend (101001 → 010110).
2. Add it to the minuend: 010110 + 010110 = 101100.
3. No end carry is obtained, so the result is negative.
4. Take 1's complement of the sum to get magnitude: 101100 → 010011

Final Answer: **$-(010011)_2$**

Explanation:

No end carry means the answer is negative, so we complement the result.

ADITYA UNIVERSITY

Binary Subtraction using 2's complement

Binary Subtraction using 2's Complement

Positive Result

Problem: $(110010)_2 - (001101)_2$

Solution:

1. Take 2's complement of the subtrahend:
   - 1's complement of 001101 = 110010
   - Add 1 → 110011
2. Add to the minuend: 110010 + 110011 = 1 100101.
3. Discard the carry.

Final Answer: **$(100101)_2$**

Explanation:

In 2's complement, carry is discarded and the result is positive.

Binary Subtraction using 2's Complement

Negative Result

Problem: $(010101)_2 - (101110)_2$

Solution:

1. Take 2's complement of the subtrahend:
   - 1's complement of 101110 = 010001
   - Add 1 → 010010
2. Add to the minuend: 010101 + 010010 = 100111.
3. No carry → result is negative.
4. Take 2's complement of 100111 to get magnitude:
   - 1's complement = 011000
   - Add 1 → 011001

Final Answer: **$-(011001)_2$**

Explanation:

Absence of carry indicates a negative result in 2's complement.

ADITYA UNIVERSITY

Decimal Subtraction using complements

- **Decimal Subtraction (Using 9's Complement)**
  - Subtract $(7254)_{10} - (3867)_{10}$ using **9's complement**
    Answer: **3387**
  - Subtract $(3867)_{10} - (7254)_{10}$ using **9's complement**
    Answer: **−3387**

- **Decimal Subtraction (Using 10's Complement)**
  - Subtract $(8532)_{10} - (4769)_{10}$ using **10's complement**
    Answer: **3763**
  - Subtract $(4769)_{10} - (8532)_{10}$ using **10's complement**
    Answer: **−3763**

Binary Addition using 2's complement

- Binary addition is performed as follows: Add the two numbers, including their sign bits, and discard any carry out of the sign (leftmost) bit position.
- Numerical examples for addition are shown below. Note that negative numbers must initially be in 2's complement form and that if the sum obtained after the addition is negative, it is in 2's complement form.

+6	00000110	−6	11111010
+13	00001101	+13	00001101
+19	00010011	+7	00000111
+6	00000110	−6	11111010
−13	11110011	−13	11110011
−7	11111001	−19	11101101

- To determine the value of a negative number when in signed-2's complement, it is necessary to convert it to a positive number to place it in a more familiar form.
- For example, the signed binary number 1111001 is negative because the leftmost bit is 1. Its 2's complement is 0000111, which is the binary equivalent of +7. We therefore recognize the original negative number to be equal to −7.

ADITYA UNIVERSITY

Binary Addition using 2's complement

Fixed-Point Representation	Floating-Point Representation
Binary point position is fixed	Binary point position can move
Represents numbers as integers or fixed fractions	Represents numbers using mantissa and exponent
Limited range of values	Very large range of values
Simple hardware implementation	Complex hardware implementation
Faster arithmetic operations	Slower compared to fixed-point
Less memory required	More memory required
Precision is fixed	Precision varies with exponent
Suitable for embedded and real-time systems	Suitable for scientific and high-precision applications
Easy to implement and understand	Difficult to implement and understand
Example: $10110. = 22_{10}$	Example: 1.011×2^4

ADITYA UNIVERSITY

Fixed-Point Representation – Examples

Example 1:

Binary number: **11001**

- Binary point is fixed at the right.
- This represents an integer value.

Decimal value:

$11001_2 = \mathbf{25_{10}}$

Example 2:

Binary number: **.1011**

- Binary point is fixed at the left.
- This represents a fractional value.

Decimal value:

$.1011_2 =$

$1\times2^{-1} + 0\times2^{-2} + 1\times2^{-3} + 1\times2^{-4}$

$= \mathbf{0.6875_{10}}$

Example 3

Binary number: **10110.11101**

- Binary point is fixed.
- Left side → integer part
- Right side → fractional part

Integer part:

$10110_2 = \mathbf{22}$

Fractional part:

$.11101_2 = \mathbf{0.90625}$

Final value:

$\mathbf{10110.11101_2 = 22.90625_{10}}$

Floating-Point Representation – Examples

Example 1:

Binary number: **11001**

Floating-point form:

$\mathbf{1.1001 \times 2^4}$

Example 2:

Binary number: **.1011**

Floating-point form:

$\mathbf{1.011 \times 2^{-1}}$

Example 3 (Your example):

Binary number: **10110.11101**

Normalized floating-point form:

$\mathbf{1.011011101 \times 2^4}$

- Mantissa = 1.011011101
- Exponent = 4

Example on Decimal:

Decimal number: **425.411**

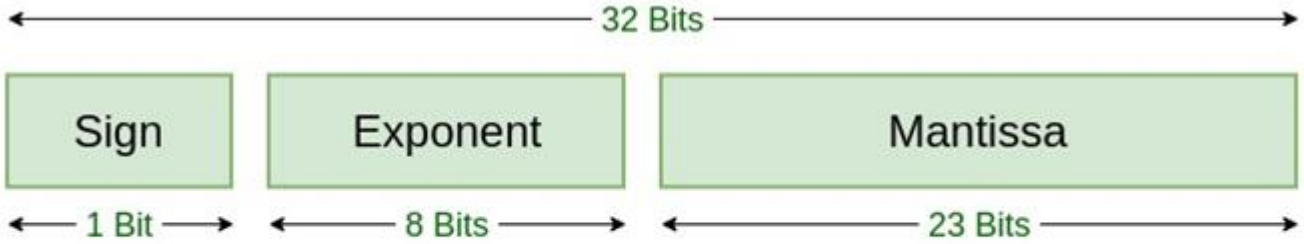
Floating-point form:

$\mathbf{4.25411 \times 10^2}$

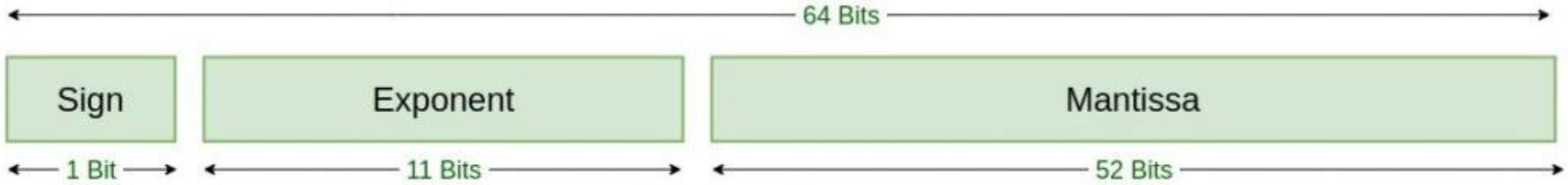
ADITYA UNIVERSITY

IEEE Standard 754 Floating Point Formats

- There are two IEEE standard 754 floating point formats:
  1. Single precision floating point representation (32-bit)
  2. Double precision floating point representation (64-bit)



Single Precision
IEEE 754 Floating-Point Standard



Double Precision
IEEE 754 Floating-Point Standard

ADITYA UNIVERSITY

Binary Codes

1. BCD (8421) Code

- BCD stands for **Binary Coded Decimal**.
- Each decimal digit is represented separately using **4 binary bits**.
- The weights of the bits are **8, 4, 2, and 1**, so it is called **8421 code**.
- It is a **weighted code**.

Example:

Decimal digit **7**

8421 BCD representation → **0111**

Another Example:

Decimal number **59**

5 → 0101

9 → 1001

BCD representation → **0101 1001**

2. 2421 Code

- 2421 code is a **weighted binary code**.
- The weights of the four bits are **2, 4, 2, and 1**.
- It is a **self-complementing code**, meaning the 9's complement can be obtained by bit complementation.

Example:

Decimal digit **4**

2421 code → **0100**

Another Example:

Decimal digit **7**

2421 code → **1101**

ADITYA UNIVERSITY

Binary Codes

3. Excess-3 Code

- Excess-3 code is a **non-weighted code**.
- It is obtained by **adding 3 to the decimal digit** and then converting the result into binary.
- It is also a **self-complementing code**.

Steps to generate Excess-3 code:

1. Add 3 to the decimal digit.
2. Convert the result into 4-bit binary.

Example:

Decimal digit **5**

5 + 3 = 8

Binary of 8 → **1000**

Another Example:

Decimal number **29**

2 → 2 + 3 = 5 → 0101

9 → 9 + 3 = 12 → 1100

Excess-3 code → **0101 1100**

ADITYA UNIVERSITY

Binary Codes

4. Gray Code

- Gray code is a **non-weighted code**.
- It is also called **Unit Distance Code**.
- Only **one bit changes between consecutive numbers**, which reduces errors.

Binary to Gray Code rule:

- MSB of Gray = MSB of Binary
- Remaining bits = XOR of adjacent binary bits

5. Excess-3 Gray Code

- Excess-3 Gray code is obtained by **first converting a decimal digit into Excess-3 code** and then **converting that binary number into Gray code**.
- It combines the advantages of **Excess-3** and **Gray code**.

Steps:

1. Convert decimal digit to Excess-3 code.
2. Convert Excess-3 binary to Gray code.

ADITYA UNIVERSITY

Gray Code

Binary to Gray Conversion

	4-bit Binary
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111

	4- bit Gray Code
0	0000
1	0001
2	0011
3	0010
4	0110
5	0111
6	0101
7	0100
8	1100
9	1101
10	1111
11	1110
12	1010
13	1011
14	1001
15	1000

Binary

B_3	B_2	B_1	B_0
1	0	1	0

Gray

G_3	G_2	G_1	G_0
1	1	1	1

$G_3 = B_3 = 1$

$G_2 = B_3 \oplus B_2 = 1 \oplus 0 = 1$

$G_1 = B_2 \oplus B_1 = 0 \oplus 1 = 1$

$G_0 = B_1 \oplus B_0 = 1 \oplus 0 = 1$

ADITYA UNIVERSITY

Gray Code

Gray to Binary Conversion

	4-bit Binary	4- bit Gray Code
0	0000	0000
1	0001	0001
2	0010	0011
3	0011	0010
4	0100	0110
5	0101	0111
6	0110	0101
7	0111	0100
8	1000	1100
9	1001	1101
10	1010	1111
11	1011	1110
12	1100	1010
13	1101	1011
14	1110	1001
15	1111	1000

Gray

G_3	G_2	G_1	G_0
1	1	1	0

Binary

B_3	B_2	B_1	B_0
1	0	1	1

$B_3 = G_3 = 1$

$B_2 = B_3 \oplus G_2 = 1 \oplus 1 = 0$

$B_1 = B_2 \oplus G_1 = 0 \oplus 1 = 1$

$B_0 = B_1 \oplus G_0 = 1 \oplus 0 = 1$

ADITYA UNIVERSITY

Binary Codes

TABLE Four Different Binary Codes for the Decimal Digit

Decimal digit	BCD 8421	2421	Excess-3	Excess-3 gray
0	0000	0000	0011	0010
1	0001	0001	0100	0110
2	0010	0010	0101	0111
3	0011	0011	0110	0101
4	0100	0100	0111	0100
5	0101	1011	1000	1100
6	0110	1100	1001	1101
7	0111	1101	1010	1111
8	1000	1110	1011	1110
9	1001	1111	1100	1010

ADITYA UNIVERSITY

Error Detection Codes

Error Detection Codes

- Binary data transmitted through a communication medium may be affected by **noise**.
- Noise can change bits from **1 to 0** or **0 to 1**.
- An **error detection code** is used to **detect errors** during data transmission.
- Detected errors **cannot be corrected**, but their presence can be identified.
- If an error is detected, the data is usually **retransmitted**.
- If errors occur too frequently, the system is checked for **malfunction**.

Parity Bit

- The **parity bit** is the most commonly used error detection method.
- A parity bit is an **extra bit** added to the binary message.
- It makes the total number of 1's either **even** or **odd**.
- Two types of parity:

  Even parity

  Odd parity

ADITYA UNIVERSITY

Parity Bit Error Detection Method

Even Parity

- The parity bit is chosen so that the **total number of 1's is even**.
- If the sum of 1's in the message is already even, parity bit = 0.
- If the sum of 1's is odd, parity bit = 1.

Odd Parity

- The parity bit is chosen so that the **total number of 1's is odd**.
- If the sum of 1's in the message is already odd, parity bit = 0.
- If the sum of 1's is even, parity bit = 1.
- The odd-parity bit is the **complement of the even-parity bit**.

Parity Generation and Checking

- At the **sending end**, the message bits are applied to a **parity generator**.
- The parity generator produces the required parity bit (odd or even).
- The message along with the parity bit is transmitted.
- At the **receiving end**, all received bits are applied to a **parity checker**.
- The parity checker verifies whether the received parity matches the adopted scheme.
- If parity does not match, an **error is detected**.

Error Detection using Odd Parity Bit

Figure Error detection with odd parity bit.

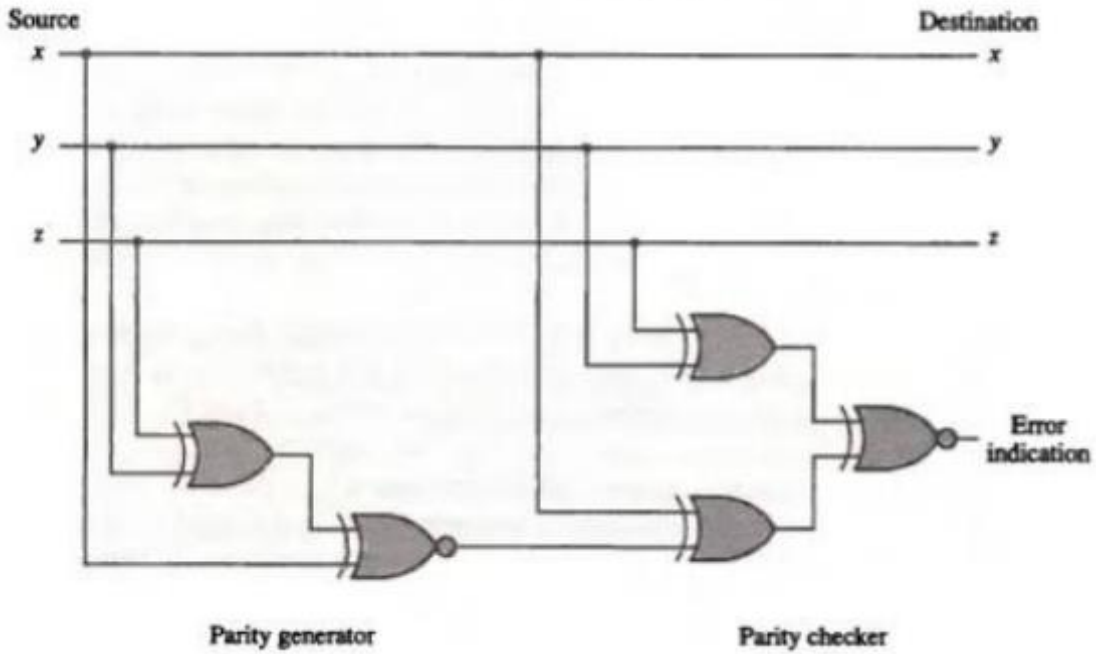


TABLE Parity Bit Generation

Message xyz	P(odd)	P(even)
000	1	0
001	0	1
010	0	1
011	1	0
100	0	1
101	1	0
110	1	0
111	0	1

Operation:

- Consider a 3-bit message (x, y, z) to be transmitted.
- At the sending end, these bits are applied to a **parity generator** made using XOR gates.
- The generator produces one extra bit called the **odd parity bit (P)**.
- This parity bit is selected so that the total number of 1's in (x, y, z, P) becomes **odd**.
- The message along with the parity bit is transmitted through the communication channel.
- At the receiving end, the received bits are applied to a **parity checker**.
- The checker again counts the total number of 1's using XOR logic.
- If the total number of 1's is **odd**, it means no error has occurred.
- If the total number of 1's is **even**, it indicates that an error has occurred during transmission.

ADITYA UNIVERSITY

Addition and Subtraction with Signed-Magnitude Data

- When two signed numbers are added or subtracted, the operation depends on their **signs**.
- Let the magnitudes of two numbers be represented by **A** and **B**.
- There are different possible cases based on whether the numbers are positive or negative.
- The required operation is performed only on the **magnitudes**, and the sign is decided separately.

TABLE Addition and Subtraction of Signed-Magnitude Numbers

Operation	Add Magnitudes	Subtract Magnitudes		
		When $A > B$	When $A < B$	When $A = B$
$(+A) + (+B)$	$+(A + B)$			
$(+A) + (-B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(-A) + (+B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$
$(-A) + (-B)$	$-(A + B)$			
$(+A) - (+B)$		$+(A - B)$	$-(B - A)$	$+(A - B)$
$(+A) - (-B)$	$+(A + B)$			
$(-A) - (+B)$	$-(A + B)$			
$(-A) - (-B)$		$-(A - B)$	$+(B - A)$	$+(A - B)$

ADITYA UNIVERSITY

Hardware for Addition and Subtraction with Signed-Magnitude Data

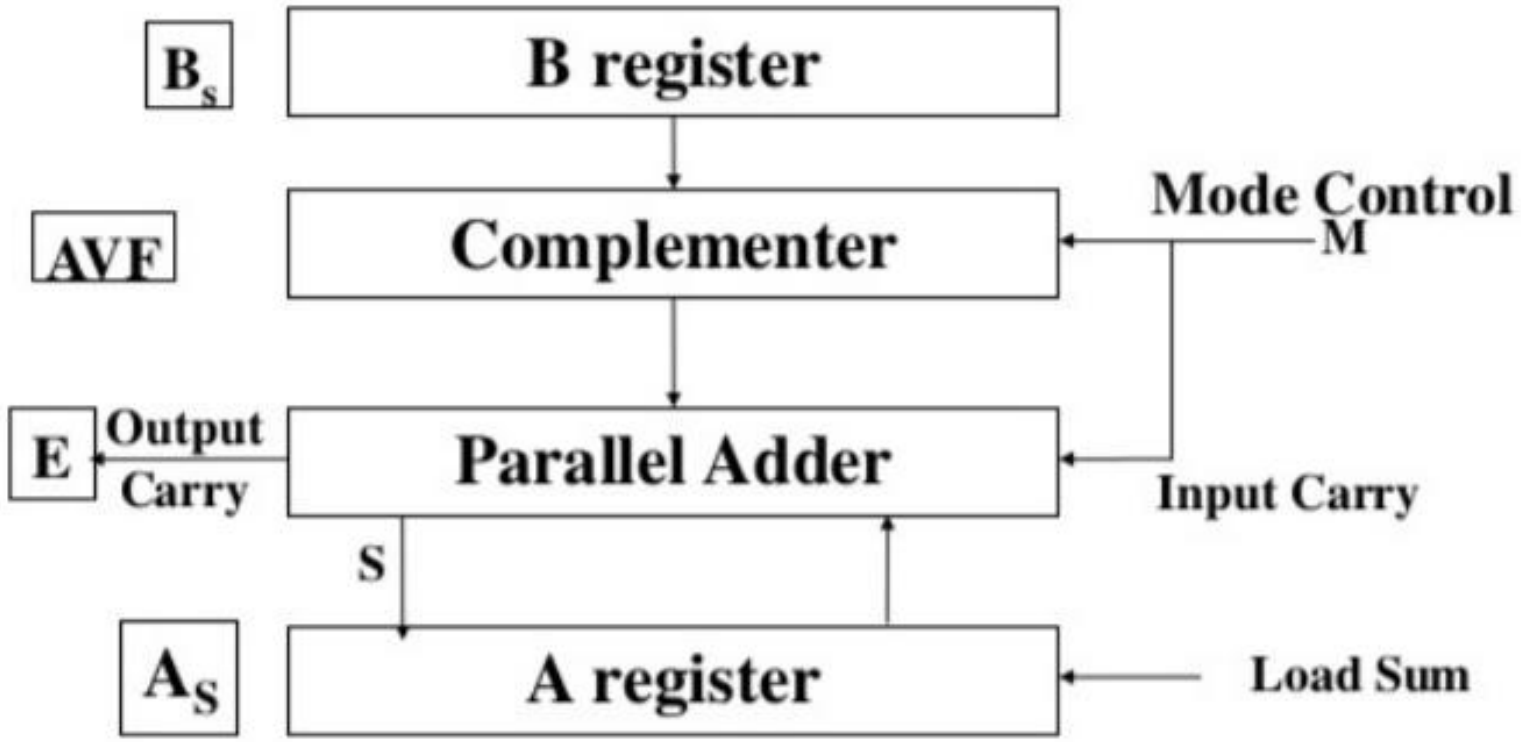
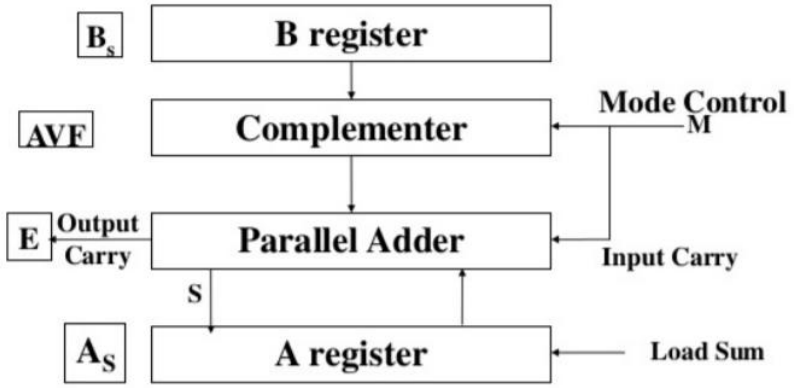


Figure: Hardware of signed-magnitude addition and subtraction

Operation of Addition and Subtraction with Signed-Magnitude Data

Basic Idea:

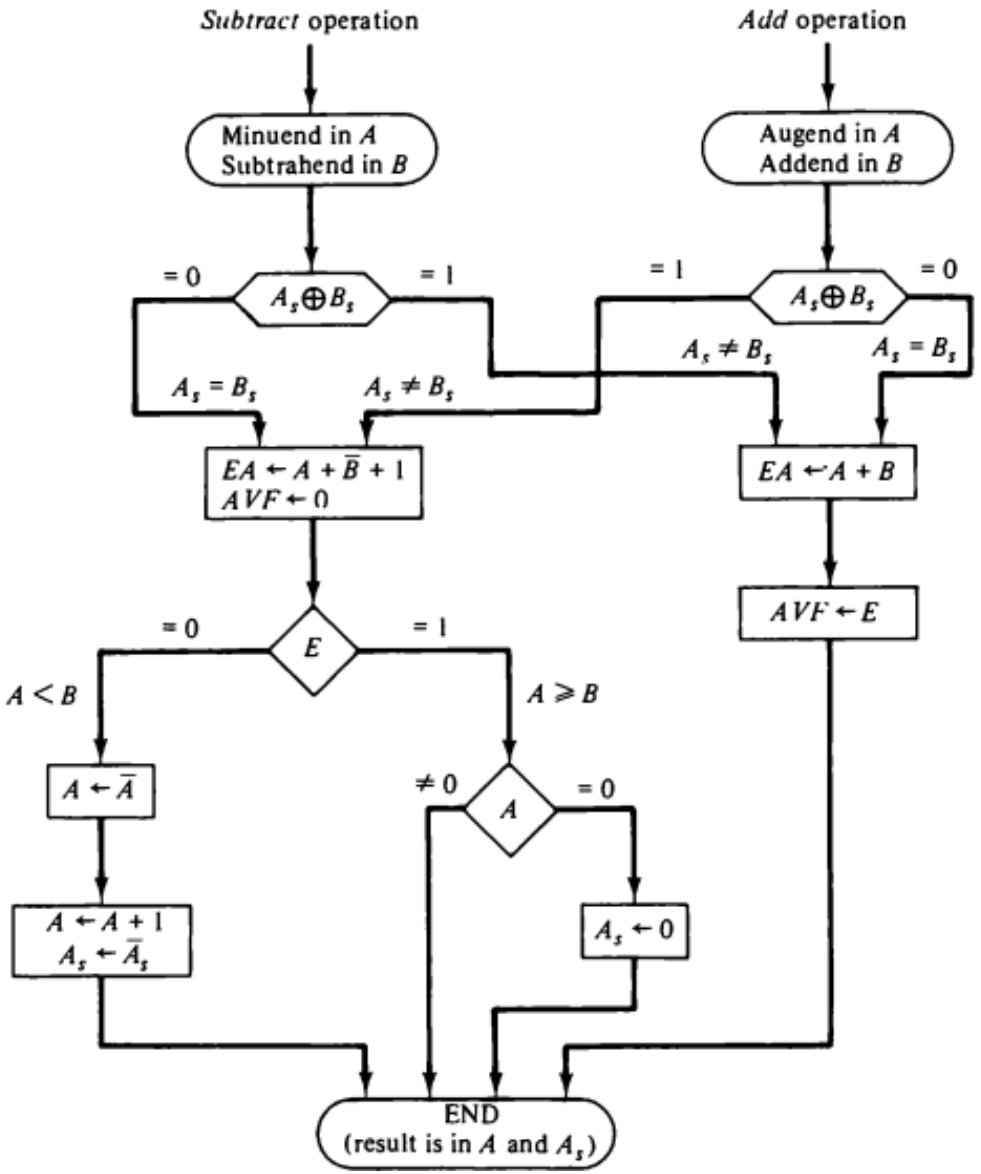
- This hardware is used to perform addition and subtraction of signed-magnitude numbers by operating on their magnitudes and deciding the sign separately.



Simple Working of the Hardware:

- The magnitudes of A and B are stored in A and B registers.
- The mode control (M) decides whether to add or subtract.
  If **M = 0** → pass B as it is (for addition).
  If **M = 1** → complement B (for subtraction of magnitudes).
- If subtraction is required, B is complemented using the complementer.
- The parallel adder adds A and B (or complemented B) and it takes an **input carry** and produces an **output carry**.
- The result is stored back in the A register.
- The E (Carry Flip-Flop) stores the carry or borrow produced during addition or subtraction which helps in determining the correct magnitude and sign of the result.
- During signed-magnitude addition or subtraction, the adder generates the result from the magnitudes, and if this result exceeds the allowed bit size an overflow occurs, which is indicated and stored by the AVF (Arithmetic Overflow Flip-Flop).

Flowchart/Algorithm for Addition & Subtraction with Signed-Magnitude



- The flowchart shows how a computer performs addition and subtraction using signed-magnitude numbers stored in registers A and B.
- First, the sign bits of the two numbers are checked.
- If both numbers have the same sign, their magnitudes are added and the result keeps the same sign.
- If the signs are different, the magnitudes are subtracted and the sign of the larger magnitude is taken as the result sign.
- During subtraction, the complement of one number is used and the carry flip-flop (E) helps in deciding which magnitude is larger.
- If the result becomes zero, the sign is set to +0.
- The AVF flip-flop indicates if overflow occurs during addition.
- Finally, the result magnitude is stored in register A and the sign bit is stored in A_s.

Figure: Flow Chart for Addition and Subtraction

2's complement addition and subtraction

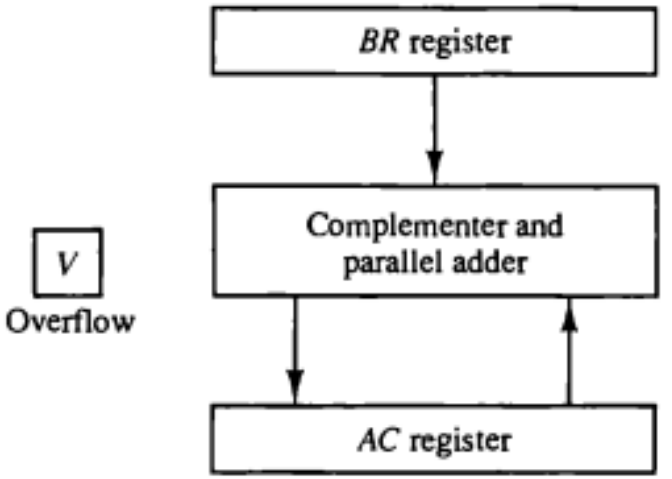


Fig. Hardware

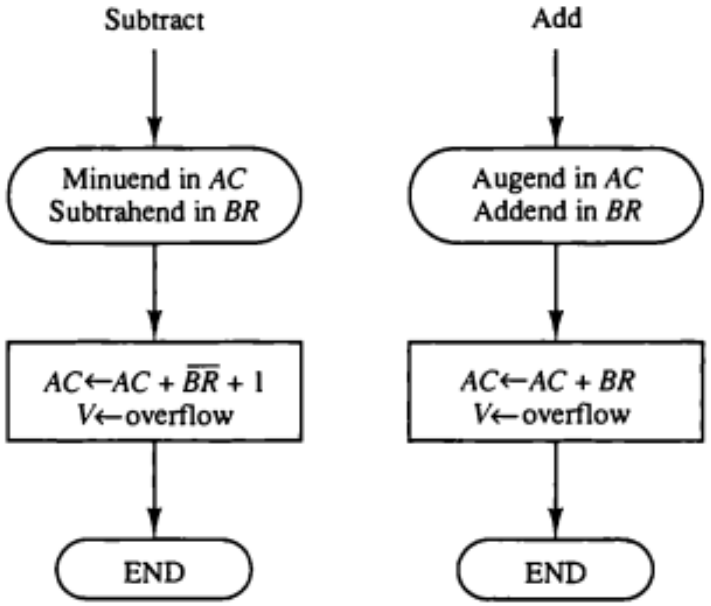


Fig: Algorithm

Hardware:

In the hardware, one number is stored in the AC register and the other in the BR register. These values are sent to the complementer and parallel adder, which performs the operation. The result is stored back in the AC register, and the overflow flag (V) indicates if the result is beyond the allowed range.

Algorithm:

According to the algorithm:

For Addition: The augend is placed in AC and the addend in BR. The hardware directly adds them using the parallel adder:

$AC \leftarrow AC + BR$

The result is stored in AC and overflow is checked.

For Subtraction: The minuend is placed in AC and the subtrahend in BR. The hardware first takes the 2's complement of BR using the complementer (complement + 1), and then adds it to AC:

$AC \leftarrow AC + (\overline{BR} + 1)$

This performs subtraction, and the result is stored in AC. Overflow is again checked.

2's complement addition and subtraction

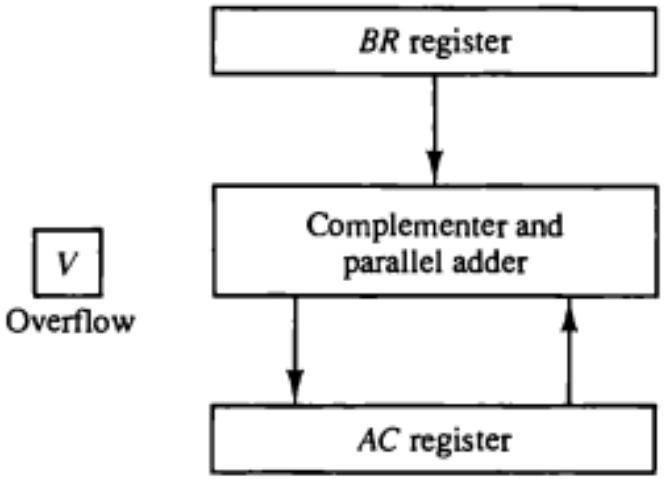


Fig. Hardware

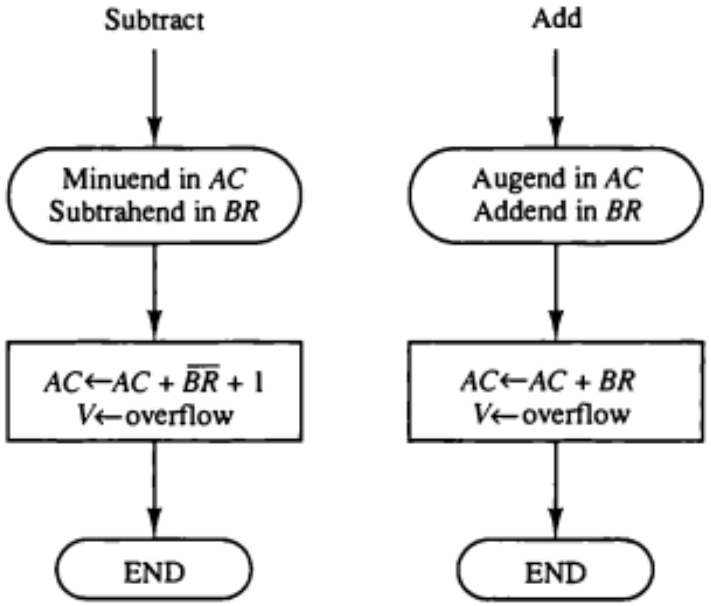


Fig: Algorithm

Hardware:

In the hardware, one number is stored in the AC register and the other in the BR register. These values are sent to the complementer and parallel adder, which performs the operation. The result is stored back in the AC register, and the overflow flag (V) indicates if the result is beyond the allowed range.

Algorithm:

According to the algorithm:

For Addition: The augend is placed in AC and the addend in BR. The hardware directly adds them using the parallel adder:

$AC \leftarrow AC + BR$

The result is stored in AC and overflow is checked.

For Subtraction: The minuend is placed in AC and the subtrahend in BR. The hardware first takes the 2's complement of BR using the complementer (complement + 1), and then adds it to AC:

$AC \leftarrow AC + (\overline{BR} + 1)$

This performs subtraction, and the result is stored in AC. Overflow is again checked.

ADITYA UNIVERSITY

Binary Multipliers - Classification

Classification based on **functionality:** Binary Multipliers are two types:

- **Unsigned Binary Multipliers:** It multiplies two Unsigned binary numbers.

  Example: Array Multiplier, Sequential Multiplier

- **Signed Binary Multipliers:** It multiplies two Signed binary numbers.

  Example: Booth's Multiplier

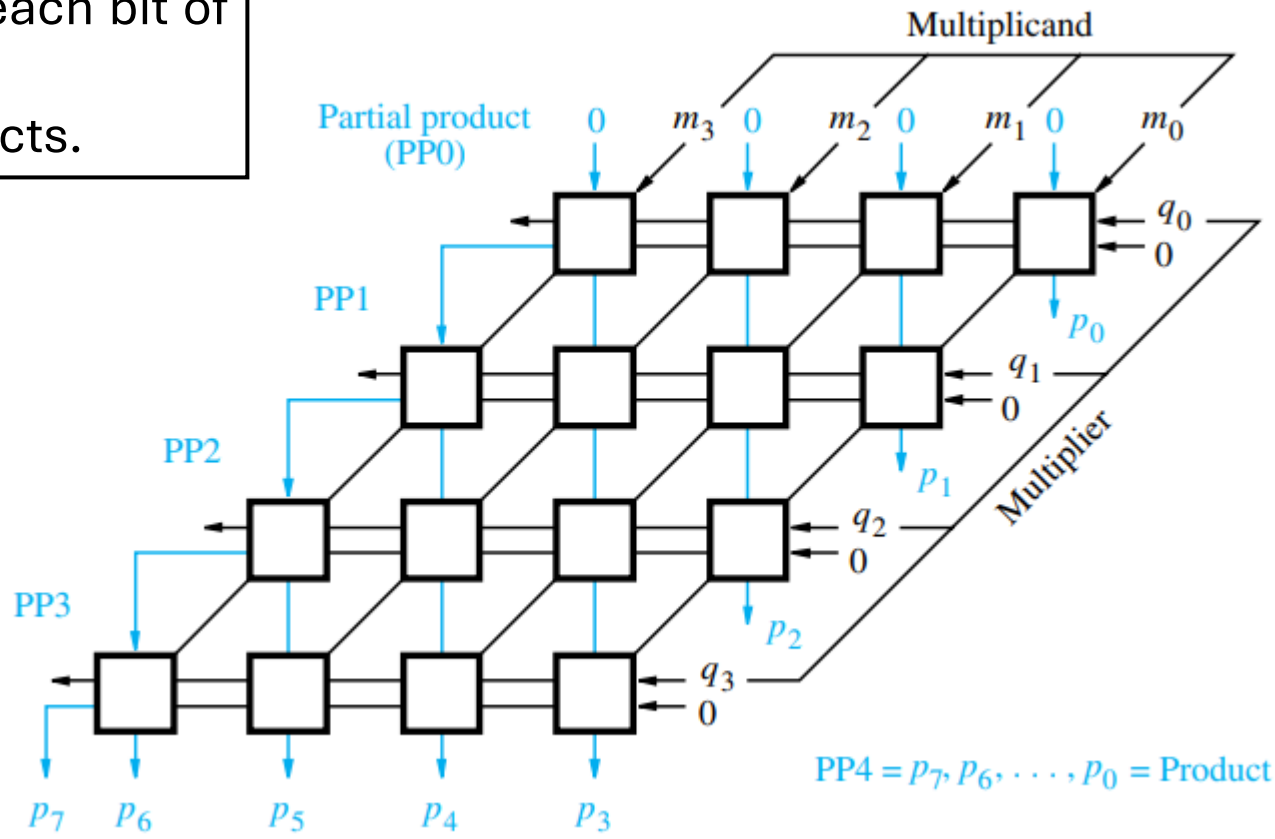
Multiplication of Unsigned Numbers-Array Multiplier

- An **Array Multiplier** is a combinational unsigned binary multiplier that can multiply two unsigned binary numbers using a grid of **AND gates** and **adders**.
- In which, the multiplication is just a series of bitwise **ANDs** (like partial products) followed by binary additions. An array multiplier does this in a grid-like layout.

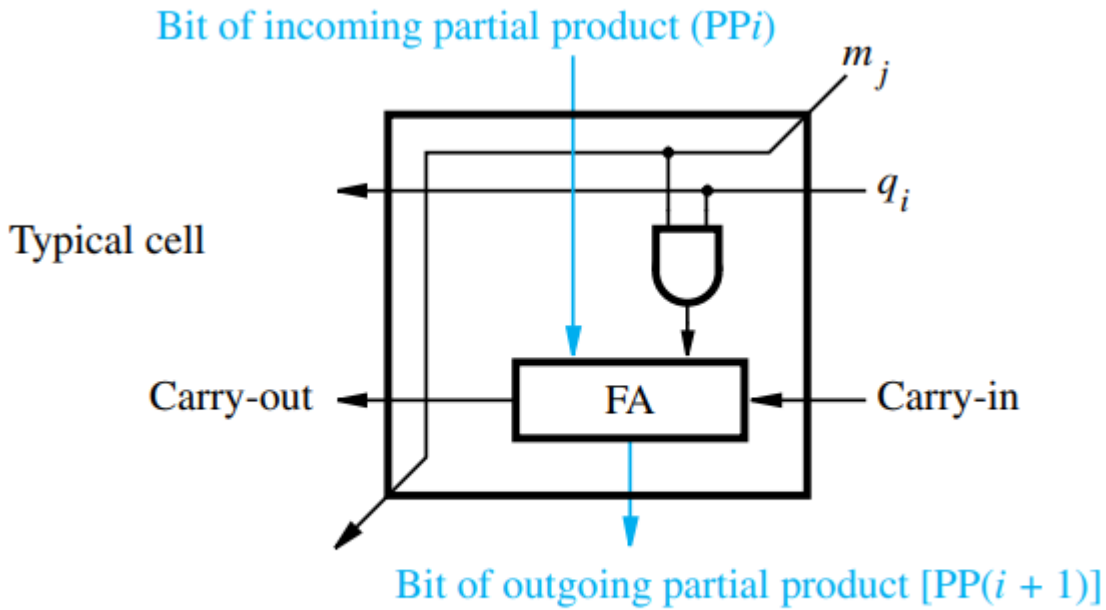
- **Rows =** Partial products (formed by ANDing each bit of the multiplier with all bits of the multiplicand).
- **Columns =** Adders to sum those partial products.

			1	1	0	1		(13) Multiplicand M
		×	1	0	1	1		(11) Multiplier Q
			1	1	0	1		
		1	1	0	1			
	0	0	0	0				
1	1	0	1					
1	0	0	0	1	1	1	1	(143) Product P

(a) Manual multiplication algorithm



Multiplication of Unsigned Numbers-Array Multiplier



(b) Array implementation

- The Fig.(b) shows a typical cell of an array multiplier, it consists of **AND gate** and **Full Adder.**
- The **AND gate** generates partial Product (PP) for the bit's q_i and m_j.
- The **Full adder** add PP, carry-in and PP coming from top Cell and produces Carry-out and PP for below Cell.

Signed Multiplier - Booth Multiplier Algorithm

The Booth Algorithm is an efficient method for multiplying two signed binary numbers in 2's complement representation.

1. Initialization

- **A (Accumulator)** = 0 (n bits)
- **Q (Multiplier)** = Given multiplier (n bits)
- **Q_{-1} (Extra bit)** = 0 (initialized to 0)
- **M (Multiplicand)** = Given multiplicand (n bits)
- **Count (n)** = Number of bits in Q.

2. Repeated Steps (for n iterations):

For each iteration, check the last bit of **Q** and extra bit (Q_0 and Q_{-1}) and take the actions as follows:

Q_0	Q_{-1}	Action
0	0	**Arithmetic Right-shift** (A, Q, Q_{-1})
0	1	**A = A + M (Addition)**, then right-shift
1	0	**A = A – M (Subtraction)**, then right-shift
1	1	**Arithmetic Right-shift** (A, Q, Q_{-1})

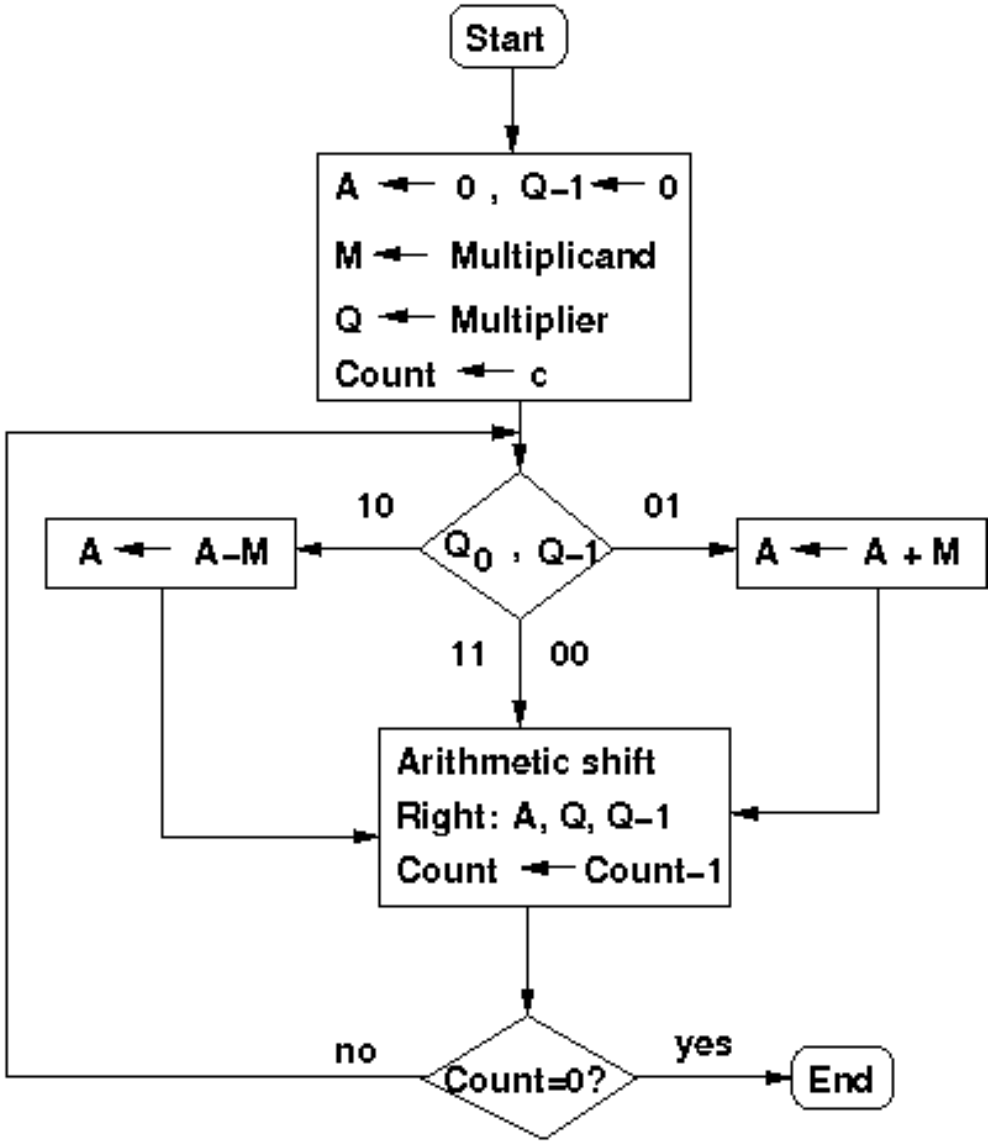
Booth Multiplier Algorithm -Explanation

Booth Algorithm Explanation:

- The multiplicand and multiplier are placed in the **m** and **Q** registers respectively.
- A 1-bit register is placed logically to the right of the LSB (least significant bit) **Q0** of **Q** register. This is denoted by **Q-1. A** and **Q-1** are initially set to 0.
- Control logic checks the two bits Q0 and Q-1. If the two bits are same (00 or 11) then all of the bits of A, Q, Q-1 are shifted 1 bit to the right.
- If they are not the same and if the combination is 10 then the multiplicand is subtracted from A and if the combination is 01 then the multiplicand is added with A. In both the cases results are stored in A, and after the addition or subtraction operation, A, Q, Q-1 are right shifted.
- The shifting is the arithmetic right shift operation where the left most bit namely, A_{n-1} is not only shifted into A_{n-2} but also remains in A_{n-1}. This is to preserve the sign of the number in A and Q.
- The result of the multiplication will appear in the A and Q.

Booth Multiplier Algorithm

Booth Multiplication – Flow Chart



Booth Multiplier Algorithm-Example

ADITYA UNIVERSITY

Step 1: Initialization:

- **Multiplicand (M) = 6 = 0110**
- **Multiplier (Q) = -3 = 1101** (in 4-bit 2's complement)
- **Q-1 = 0** (extra bit initialized to 0)
- **A = 0000** (accumulator)
- Number of bits (**n**) = 4 (because we use 4-bit numbers)

Step 3: Final Result

- After 4 cycles, the combined A and Q registers give the product: 11101110_2 (in 8-bit 2's complement).
- Converting back to decimal:
  - The leftmost bit is **1**, so it's negative.
  - Take 2's complement:
    Invert $11101110 \rightarrow 00010001$
    Add $1 \rightarrow 00010010 = 18_{10}$.
  - Thus, $11101110_2 = -18_{10}$.

Step 2: Algorithm steps:

Cycle	A	Q	Q_{-1}	Operation
0	0000	1101	0	Initialization
1	0000	1101	0	$Q_0Q_{-1} = 10 \rightarrow A = A - M$
	1010	1101	0	Arithmetic Right Shift (A, Q, Q_{-1})
	1101	0110	1	
2	1101	0110	1	$Q_0Q_{-1} = 01 \rightarrow A = A + M$
	0011	0110	1	Arithmetic Right Shift (A, Q, Q_{-1})
	0001	1011	0	
3	0001	1011	0	$Q_0Q_{-1} = 10 \rightarrow A = A - M$
	1011	1011	0	Arithmetic Right Shift (A, Q, Q_{-1})
	1101	1101	1	
4	1101	1101	1	$Q_0Q_{-1} = 11 \rightarrow$ No op
	1101	1101	1	Arithmetic Right Shift (A, Q, Q_{-1})
	1110	1110	1	

Booth Multiplier Algorithm-Hardware Realization

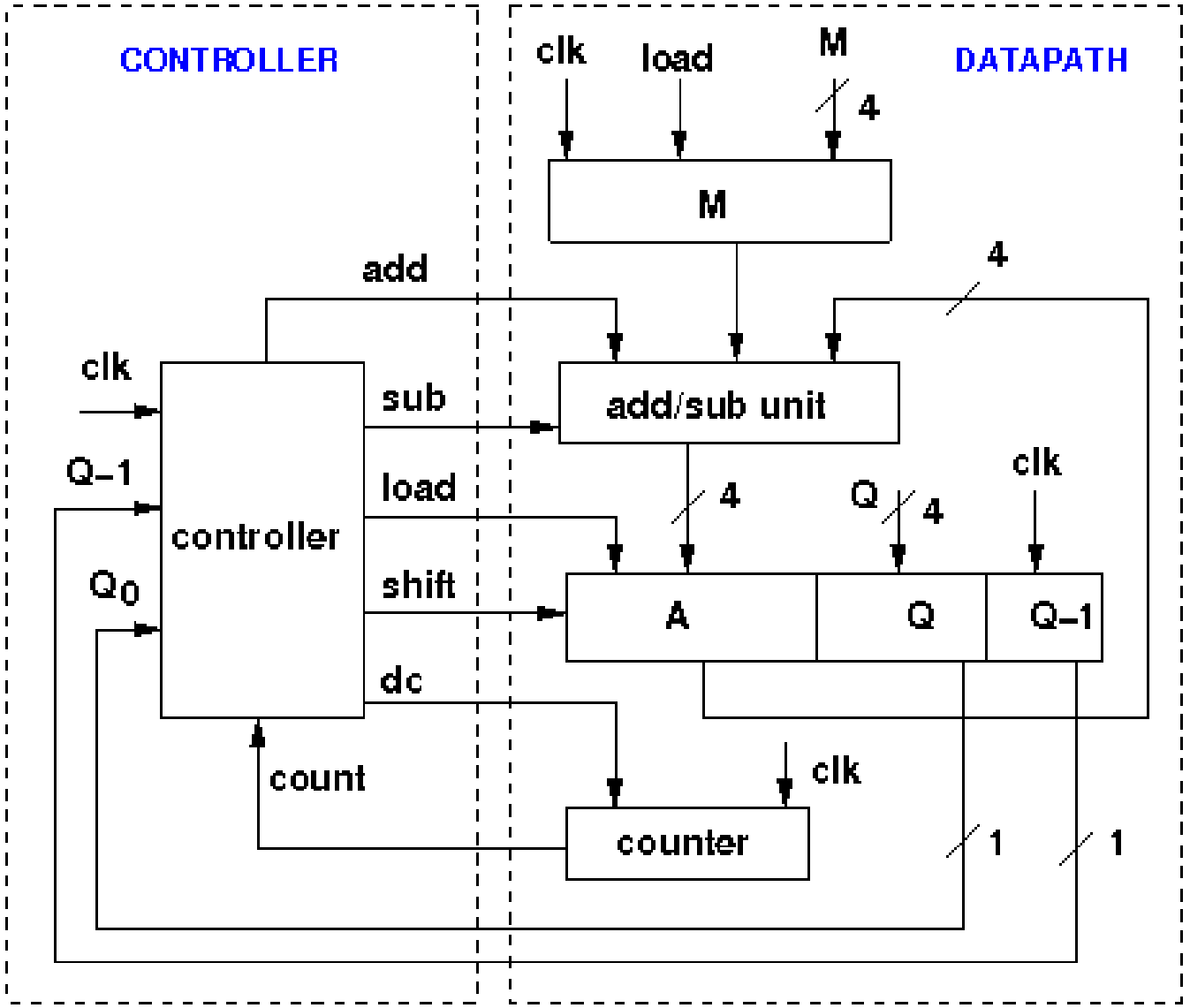


Fig. Circuit arrangement for Booth's Multiplier

Booth Multiplier Algorithm-Hardware Realization

- Booth's algorithm can be implemented in many ways. The schematic diagram of the Booth's multiplier which multiplies two 4-bit numbers shown in Figure, it is designed using a controller and a datapath. The operations on the data in the datapath is controlled by the control signal received from the controller. The data path contains registers to hold multiplier, multiplicand, intermediate results, data processing units like ALU, adder/subtractor etc., counter and other combinational units.
- M, Q, A are 4-bit and Q-1 is a 1-bit register. M holds the multiplicand, Q holds the multiplier, A holds the results of adder/subtractor unit. The counter is a down counter which counts the number of operations needed for the multiplication.
- The data flow in the data path is controlled by the five control signals generated from the controller. these signals are *load* (to load data in registers), *add* (to initiate addition operation), *sub* (to initiate subtraction operation), *shift* (to initiate arithmetic right shift operation), *dc* (this is to decrement counter). The controller generates the control signals according to the input received from the data path. Here the inputs are the least significant Q0 bit of Q register, Q-1 bit and count bit from the down counter.

Division Algorithms - Restoring Division of Binary Numbers - Introduction

A digital **division circuit** works by:

1. **Aligning the divisor** with the relevant part of the dividend.
2. Performing a **subtraction** to check if the divisor "fits".
   - If the result is **non-negative**, it keeps the result and sets the quotient bit to **1**.
   - If the result is **negative**, it **restores** the original value by adding back the divisor and sets the quotient bit to **0**.
3. The process continues by **shifting in the next bit** of the dividend.

This method is known as the **restoring division algorithm or Longhand division.**

Mathematical Example:

```
      21
13 ) 274
     26
     ---
      14
      13
     ---
       1
```

```
          10101
1101 ) 100010010
       1101
       ----
       10000
        1101
        ----
        1110
        1101
        ----
           1
```

Restoring Division of Binary Numbers-Procedure

- The logic circuit arrangement for implementation of the restoring division Algorithm in shown in Fig.
- An n-bit positive divisor is loaded into register M and an n-bit positive dividend is loaded into register Q at the start of the operation.
- Register A is set to 0. After the division is complete, the n-bit quotient is in register Q and the remainder is in register A.
- The required subtractions are facilitated by using 2's-complement arithmetic.
- The extra bit position at the left end of both A and M accommodates the sign bit during subtractions. The following algorithm performs restoring division.
- Do the following three steps n times:

  1. Shift A and Q left one bit position.

  2. Subtract M from A, and place the answer back in A.

  3. If the sign bit (MSB bit) of A is 1, set q0 to 0 and add M back to A (that is, restore A); otherwise, set q0 to 1.

Restoring Division of Binary Numbers-Procedure

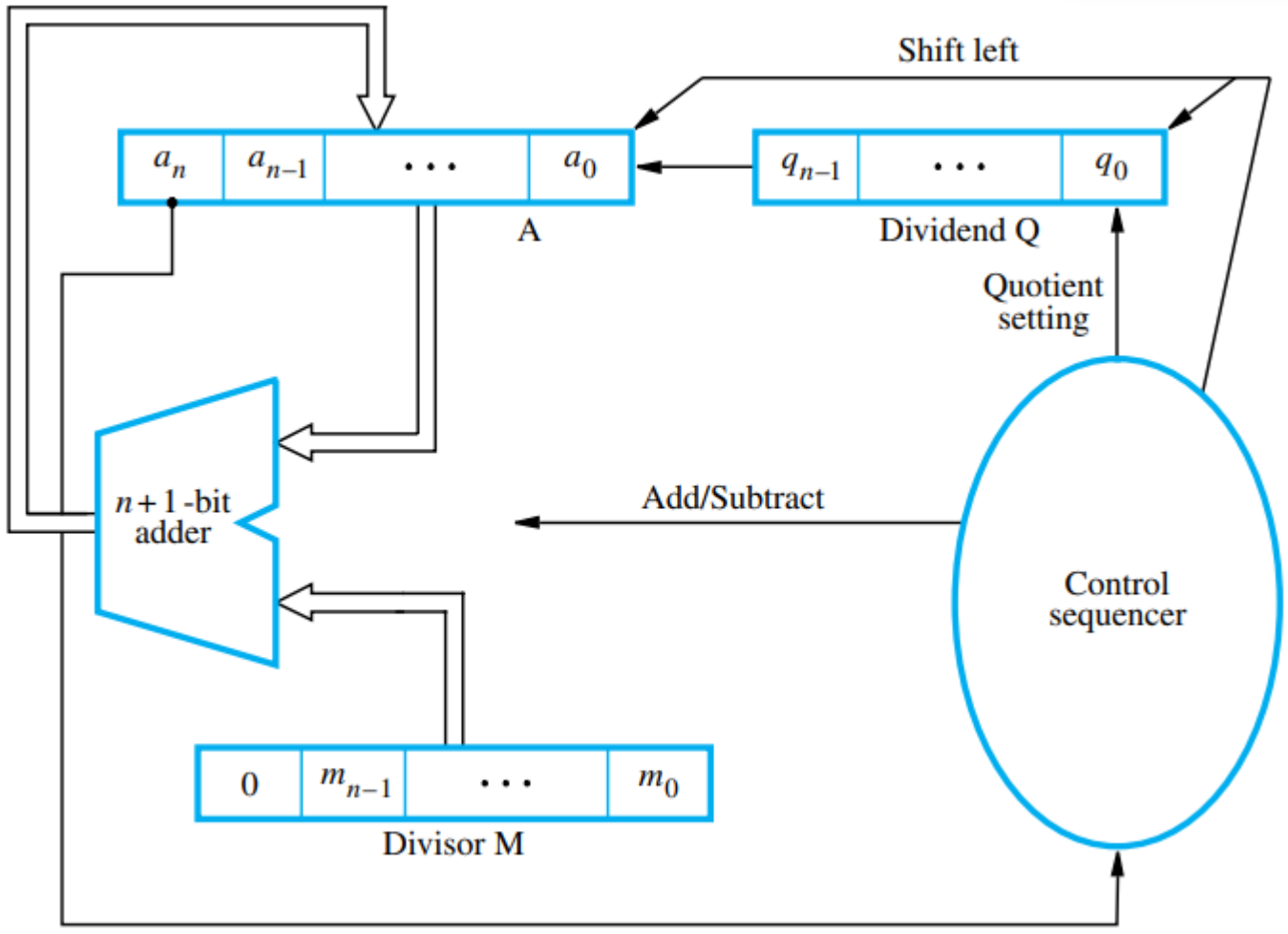


Fig. Circuit arrangement for binary division

A restoring division example:

$$\begin{array}{r} 10 \\ 11\overline{)\,1000} \\ \underline{11} \\ 10 \end{array}$$

	A	Q	
Initially	0 0 0 0 0	1 0 0 0	
M	0 0 0 1 1		
Shift	0 0 0 0 1	0 0 0 □	First cycle
Subtract	1 1 1 0 1		
Set q_0	(1) 1 1 1 0		
Restore	1 1		
	0 0 0 0 1	0 0 0 0	
Shift	0 0 0 1 0	0 0 0 □	Second cycle
Subtract	1 1 1 0 1		
Set q_0	(1) 1 1 1 1		
Restore	1 1		
	0 0 0 1 0	0 0 0 0	
Shift	0 0 1 0 0	0 0 0 □	Third cycle
Subtract	1 1 1 0 1		
Set q_0	(0) 0 0 0 1		
Shift	0 0 0 1 0	0 0 0 1	Fourth cycle
Subtract	1 1 1 0 1	0 0 1 □	
Set q_0	(1) 1 1 1 1		
Restore	1 1		
	0 0 0 1 0	0 0 1 0	
	Remainder	Quotient	